Probability Analysis and Application of XOR Difference Between (X+K)mod2^n and X⊕K

Jie Guan
Zhengzhou Information Science Technology Institute
Zhengzhou, China
guanjie007@163.com

Lin Ding
Zhengzhou Information Science Technology Institute
Zhengzhou, China
dinglin1987dl@163.com

***Abstract*—Based on the "larger probability is priory to pick up" principle in savage attack, the best algorithm with computational complexity of *O*(*n*) is given to solve the equivalent problem of (*X*+*K*)mod2^n and *X* ⊕ *K*, which improving the Guess and Determine attack on SNOW1.0 of Hawkes, the data complexity of our attack decrease to 1/32 of Hawkes's and the computational complexity keeps the same as before.**

Keywords- XOR Difference; addition on module 2^n; SNOW; Guess and Determine attack

I. INTRODUCTION

Addition on modular 2^n and bitwise addition (XOR) are two operations which are often used in cipher algorithms, the designers often use the operation addition on modular 2^n as the nonlinear part, and use XOR as the linear part in cipher algorithm ,such as SNOW [1-3] ,Py[4] and so on. Because Addition on modular 2^n and XOR are two different group operations, so using the two operations mixed is helped to increase the security strength. In many cryptanalysis technology such as distinguishing attack[5],linear attack[6]and so on, using the XOR operation as Addition on modular 2^n to linearization the nonlinear part, the leakage rule of the cipher algorithm can be get from the method.

Added constant K on modular 2^n -(*X*+*K*)mod2^n is widely used in many block ciphers, stream ciphers and hash functions. Because the input variable of this operation is only one dimension：$X \in Z/(2^n)$ (here *K* is denoted as a fixed constant),so the cryptography property is different from addition on modular 2^n with multi-variable. Now the differential property and linear property of the operation (*X*+*K*)mod2^n were published in references [6-8], but the XOR difference probability analysis of (*X*+*K*)mod2^n and *X*⊕*K* is not seen by now.

In this paper, the structure and count problems of XOR difference probability of (*X*+*K*)mod2^n and *X*⊕*K* are solved completely for some special K, the best algorithm with computational complexity of *O*(*n*) is given to solve the equivalent problem of the two operations based on the "larger probability is priory to pick up" principle in savage attack, then we improve the Guess and Determine attack on SNOW1.0 of Hawkes[10] based on the algorithm, the data complexity of our attack decrease to 1/32 of Hawkes's, and the computational complexity keeps the same as before.

978-1-4244-6943-7/10/$26.00 ©2010 IEEE

II. EQUIVALENT PROBLEM OF (*X*+*K*)MOD2^N AND *X*⊕*K*

The following notations are used throughout this paper.

$\forall X \in Z/(2^n)$, note $X = \sum_{i=0}^{n-1} x_i 2^{i-1}$ and $x_i \in \{0,1\}$,then let X=$(x_{n-1}, x_{n-2}, \cdots, x_0)$,where x_i is the i th bit of X ,x_0 is the least-significant bit of X, and $x_{n-1}$is the most-significant bit of X.

Let $Y = (X+K) \bmod 2^n$,then $\forall i: 0 \le i \le n-1$,let $y_i = x_i \oplus k_i \oplus \lambda_i$,here $\lambda_i = x_{i-1}k_{i-1} \oplus x_{i-1}\lambda_{i-1} \oplus k_{i-1}\lambda_{i-1}$,denote the carry bit from the i-1th bit to the ith bit of Y, and $\lambda_0 = 0$.

The equivalent problem of (*X*+*K*)mod2^n and *X* ⊕ *K* means：for a fixed unknown constant K, how to choose λ to make the (1)come into existence :

$$(X+K)\bmod 2^n = X \oplus K \oplus \lambda \qquad (1).$$

That is to say, how to confirm the value of XOR difference $\lambda = (\lambda_{n-1}, \lambda_{n-2}, \cdots, \lambda_0)$, so that $X \oplus K \oplus \lambda$ can take the place of $(X+K) \bmod 2^n$.

We study the probability rule of the difference value λ =(*X*+*K*)mod2^n ⊕ *X*⊕*K* for some special K in the following sections.

A. The structure and count of the XOR difference

When $K \in A_i = \{2^i, 2^{n-1}+2^i\}$ （ $0 \le i \le n-2$ ） ,the following conclusion solve the structure and count problems of XOR difference probability of (*X*+*K*)mod2^n and *X*⊕*K*.

For reason to depict easily, we give the following definition.

Definition 1 :Let $X = (x_{n-1}, x_{n-2}, \cdots, x_0)$, the succession weight of X means the maximal numbers which take value 1 successionally from the least to the most significant bit of X, denoted by $\overline{w(X)}$ .Viz. if $x_0 = x_1 = \cdots = x_t = 1, x_{t+1} = 0$, then $\overline{w(X)} = t+1$;if x_0=0, the note $\overline{w(X)} = 0$.

Let $X^{(i)} = (x_{n-1}, x_{n-2}, \cdots, x_i)$,then the succession weight of $X^{(i)}$ is the maximal numbers which take value 1 successionally from the *i*th to the most significant bit of X, denoted by $\overline{w(X^{(i)})}$.

Theorem 1: Let (*X*+*K*)mod2^n = $X \oplus K \oplus \lambda$, here $\lambda = (\lambda_{n-1}, \lambda_{n-2}, \cdots, \lambda_0)$ means the carry

vector, $\forall i: 0 \le i \le n-2$,if $K \in A_i = \{2^i, 2^{n-1}+2^i\}$,then probability distribution of the value of λ is as followed:

（1） $p\left(\overline{w\left(\lambda^{(i+1)}\right)} = t; w(\lambda) = t\right) = \frac{1}{2^{t+1}}$， $0 \le t \le n-2-i$,

$p\left(\overline{w\left(\lambda^{(i+1)}\right)} = n-i-1; w(\lambda) = n-i-1\right) = \frac{1}{2^{n-i-1}}$；

（2） $p\left(\overline{w\left(\lambda^{(i+1)}\right)} = t; w(\lambda) \ne t\right) = 0$， $0 \le t \le n-1-i$；

（3） $\sum_{t=0}^{n-i-1} p\left(\overline{w\left(\lambda^{(i+1)}\right)} = t\right) = 1$.

Proof： Because of $K \in \{2^i, 2^{n-1}+2^i\}$, so k_i=1 and $k_j = 0, j \ne i, 0 \le j \le n-2$ ， According to $(X+K)\bmod 2^n = X \oplus K \oplus \lambda$,then $\lambda_0 = \cdots = \lambda_i = 0$ .We discuss the succession weight of $X^{(i)}$, Viz. $\overline{w\left(X^{(i)}\right)}$ separately:

Case1. when $\overline{w\left(X^{(i)}\right)} = 0$， Viz. if x_i=0,because of k_i=1and $k_j = 0, j \ne i, 0 \le j \le n-2$,so $\lambda = 0$, then

$p\left(\overline{w\left(\lambda^{(i+1)}\right)} = 0; w(\lambda) = 0\right) = p(x_i = 0) = \frac{1}{2}$,and

$p\left(\overline{w\left(\lambda^{(i+1)}\right)} = 0, w(\lambda) \ne 0\right) = 0$.

Case2. when $\overline{w\left(X^{(i)}\right)} = t$,here $1 \le t \le n-2-i$ ， Viz. $x_i = x_{i+1} = \cdots = x_{i+t-1} = 1$ and $x_{i+t} = 0$ ， we can easily get $\lambda_{i+1} = \lambda_{i+2} = \cdots = \lambda_{i+t} = 1$ and all of the other bits of λ take the value 0,Viz. $\overline{w\left(\lambda^{(i+1)}\right)} = t$ and $w(\lambda) = t$,then $p\left(\overline{w\left(\lambda^{(i+1)}\right)} = t, w(\lambda) = t\right) = p\left(x_i = \cdots = x_{i+t-1} = 1, x_{i+t} = 0\right) = \frac{1}{2^{t+1}}$,and $p\left(\overline{w\left(\lambda^{(i+1)}\right)} = t, w(\lambda) \ne t\right) = 0$.

Case3. $\overline{w\left(X^{(i)}\right)} = n-1-i$ （or *n-i*),

Viz. $x_i = x_{i+1} = \cdots = x_{n-2} = 1$ and x_{n-1}=0（or 1）,whatever x_{n-1}take the value 0 or 1,we can get $\lambda_{i+1} = \lambda_{i+2} = \cdots = \lambda_{n-1} = 1$ and all of the orther bits of λ take the value 0,Viz.

$p\left(\overline{w\left(\lambda^{(i+1)}\right)} = n-i-1, w(\lambda) = n-i-1\right) = p\left(x_i = x_{i+1} = \cdots = x_{n-2} = 1\right) = \frac{1}{2^{n-i-1}}$

and $p\left(\overline{w\left(\lambda^{(i+1)}\right)} = n-i-1, w(\lambda) \ne n-i-1\right) = 0$.

Synthesize the three cases upwords, the conclusions(1) and (2) are come to existence, the conclusion (3) is proved as followed:

$$\sum_{t=0}^{n-i-1} p\left(\overline{w\left(\lambda^{(i+1)}\right)} = t\right) = \sum_{t=0}^{n-i-1}\left[p\left(\overline{w\left(\lambda^{(i+1)}\right)} = t, w(\lambda) = t\right) + p\left(\overline{w\left(\lambda^{(i+1)}\right)} = t, w(\lambda) \ne t\right)\right]$$

$$= \sum_{t=0}^{n-i-1} p\left(\overline{w\left(\lambda^{(i+1)}\right)} = t, w(\lambda) = t\right) = \frac{1}{2} + \frac{1}{4} + \cdots + \frac{1}{2^{n-i-1}} + \frac{1}{2^{n-i-1}} = 1.$$

over.

Actually, Theorem 1 points out that ,when $K \in A_i = \{2^i, 2^{n-1}+2^i\}$,the probability of (1)come to existence is only to be 0 or the power of 2,and the probability decrease exponentially according to $\overline{w\left(\lambda^{(i+1)}\right)}$ increasing （all of the other bits of λ take the value 0）.

Deduce1：When $K \in \bigcup_{i=0}^{n-2} A_i$, t_j note the number of (K, λ) which make the probability of (1)holds is equal to $2^{-j}(1 \le j \le n-1)$,Viz.

$$t_j = \#\left\{(K, \lambda): p((X+K) \bmod 2^n \oplus X \oplus K = \lambda) = \frac{1}{2^j}\right\}$$,then $t_j = n-j+1$.

Proof: From theorem 1,we can easily get it .

over.

B. Algorithm to sovle the equivalent problem of (X+K)mod2^n and $X \oplus K$

When $K \in \bigcup_{i=0}^{n-2} A_i$,From Theorem 1 we know that， as (1)holding with different probability because of different (K, λ),so the problem of the order of guessing (K, λ) should be considered, that is to say, how to arrange the order to get the minimal expectation value of guessing numbers. Based on the "larger probability is priory to pick up" principle in savage attack, we can achieve the goal of getting minimal expectation value of guessing numbers.

Lemma 1[11] Assume the order of guessing key is $k_1, k_2, \cdots, k_N$ in savage attack ,let $p_1, p_2, \cdots, p_N$ note the probability of $k_1, k_2, \cdots, k_N$ to be the right key, and $p_1 \ge p_2 \ge \cdots \ge p_N \ge 0$,so the average computational complexity of the savage attack is the minimum when the guessing order is $k_1, k_2, \cdots, k_N$.

Making the use of the "larger probability is priory to pick up" principle, an algorithm is given to solve the equivalent problem of the two operations, details are in the following:

Algorithm1: Algorithm to sovle the equivalent problem of $(X+K)\bmod 2^n$ and $X \oplus K$ when $K \in \bigcup_{i=0}^{n-2} A_i$

STEP1：Guessing $K \in A_{n-1}$ and $\lambda = 0$,give the (K, λ) to the others who can confirm the (1)to be right or not ,if the (1)is right, then turn to step *n*+1; otherwise execute the following steps。

STEP $t\left(2 \le t \le n\right)$: For i=0;i≤n-t;i++;

Guessing $K \in A_i$ and $\lambda = \left(2^{t-1} - 2\right)2^i$ .If the (1)is right, then turn to STEP n+1; otherwise i++;

If i=n-t and the (K, λ) is not right ,then guessing $K \in A_{n-t}$ and $\lambda = 2^n - 2^{n-t+1}$;if (1) is right then turn to STEPn+1, otherwise execute STEPt+1.

STEPn+1: output the (K, λ), end.

Explaination: In the algorithm, assumed the others know the value of (X, K) ,so they can confirm the (1)is right or not. In practice attacking, the work of the others can be done by adding the verified process.

Theorem 2 The Algorithm 1 is the best algorithm to solve the equivalent problem of (X+K)mod2^n and $X \oplus K$ when $K \in \bigcup_{i=0}^{n-2} A_i$,the expectation value of guessing numbers is $\frac{1}{2n}\left(3n^2 - 7n + 12 - 3 \cdot 2^{-n+3}\right)$,the computational complexity is $O(n)$,the success rate is 1.

Proof: Because the Algorithm 1 is given by Theorem 1 based on the principle of larger probability is priory to pick up, according to Lemma 1 we know that, the expectation value of guessing numbers in algorithm 1 is the minimum, so the Algorithm 1 is the best algorithm to solve the equivalent problem depicted before.

By deduce1 we know, the expectation value of guessing numbers of Algorithm 1 is

$$\frac{1}{n}\sum_{i=1}^{t_0} i + \frac{1}{2n}\sum_{i=t_0+1}^{t_0+t_1} i + \frac{1}{4n}\sum_{i=t_0+t_1+1}^{t_0+t_1+t_2} i + \cdots + \frac{1}{2^{n-1}n}\sum_{i=t_0+t_1+\cdots+t_{n-3}+1}^{t_0+t_1+\cdots+t_{n-3}+t_{n-2}+t_{n-1}} i$$

$$= \frac{1}{2n}\left(3n^2 - 7n + 12 - 3 \cdot 2^{-n+3}\right)$$

So the computational complexity of Algorithm1 is $O(n)$,according to the conclusion(3) of Theorem 1,the success rate of Algorithm1 is 1. over.

We improved the Guess and Determine attack on SNOW1.0 of Hawkes based on the algorithm, the data complexity is decreased to 1/32 of Hawkes's, computational complexity is keep the same as before.

III. IMPROVED GUESS AND DETERMINE ATTACK ON SNOW1.0

SNOW series algorithms are software-oriented stream cipher, three versions are separated SNOW1.0[1],SNOW2.0[2] and SNOW3G[3]. Hawkes and Rose presented a Guess and Determine attack on SNOW1.0 in [10], with the computational complexity of $O(2^{224})$, the data complexity of $O(2^{95})$.

A. Description of SNOW1.0

SNOW1.0 is a stream cipher designed oriented words(32 bit) operation, the key size is 128 bits or 256 bits. The Key stream Generator(KSG) consists of a length 16 Linear Feedback Register (LFSR) over $GF(2^{32})$ and a finite state machine(FSM).The FSM consists of two 32 bits registers, called R_1 and R_2,as well as a some operations to calculate the output and the next state of R_1 and R_2.The KSG is shown in Fig.1.

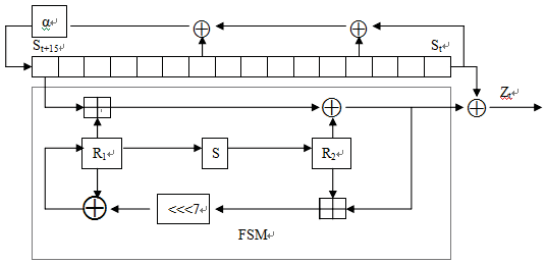

Figure 1. The KSG of SNOW1.0

In Fig.1,the addition sign " ⊕ "represents bitwise addition(XOR); "田"represents addition module 2^{32};<<<7 denotes left-rotated by 7 bits, S denotes a 32-to-32 bit S-box. The LFSR has a primitive feedback polynomial over $GF(2^{32})$ which is $p(x) = x^{16} + x^{13} + x^7 + \alpha^{-1}$,where α is a constant.

Let f_t=(s_{t+15} 田 $R1_t$) ⊕$R2_t$,then the 32 bits of the running key z_t is calculate as z_t=f_t ⊕ s_t.Inside FSM, the next state transformation about $R_1$and R_2 is as follows:

$$R1_{t+1}=((f_t \text{ 田 } R2_t)<<<7) \oplus R1_t, R2_{t+1}=S(R1_t)$$

B. Improved Guess and DetermineAttack

The improved Guess and Determine (GD) attack on SNOW1.0 is given as follows based on the attack of [10].

Assumed the attackers got some running key words $\{z_t\}$,t=1,2,…,N, the GD attack has four"phases":

Phase One: Give the assume of attack

Assume 1: $R2_t = S\left(R1_t \oplus \left(2^{32} - 1\right)\right)$

Assume 2: $R2_{t+14} = S\left(R1_{t+14} \oplus \left(2^{32} - 1\right)\right)$

Assume 3: $R1_{t+3} \in \bigcup_{i=0}^{31} A_i$

Phase Two: the guess process can be divided into the following step1-step33:

STEP1. Guesses the values of $s_t, s_{t+1}, R1_t, R1_{t+14}$ (128 bits in total) and the most -significant bit of the two different words in the set A_i .

The following 32 steps(STEP2- STEP33) is based on the Algorithm 1,where $R1_{t+3}$ is the K of Algorithm 1, every step gives the value of ($R1_{t+3}$, λ),the process of confirming the (1)which is right or not(done by the others in the Algorithm 1)can be done by the following Phase Three.

Phase Three: The values of the series of 16 states $\left(s_{t+15},\cdots,s_t\right)$ and $R1_t$,$R2_t$can be determined based on Phase One and Phase Two, the concrete determining process is the same as the attack in [10].

Phase Four: The running key can be calculated from the initial states we got in Phase three, we can verify the correctness of the result by comparing with the observed running key.

The improvement we made in this paper is increasing the range of $R1_{t+3}$,then the probability of the occurring of the attack assume is $\frac{64}{2^{96}}=2^{-90}$,so the data amount of our attack is $O(2^{90})$.However ,because of $R1_{t+3}\in\bigcup_{i=0}^{31}A_i$,so we need to confirm the values of ($R1_{t+3}$, λ) by the Algorithm 1,according to the Theorem 2,the guessing number of the Algorithm 1 is :

$$\frac{1}{2\cdot 32}\left(3\cdot 32^2-7\cdot 32+12-3\cdot 2^{-32+3}\right)=44.688$$

Then the total guessing bits of our attack is $129+\log_2(44.488)=134$,so the computational complexity of our attack is $O(2^{224})$.Compared with the attack of [10], the data complexity of our attack decreases to 1/32 of Hawkes's and the computational complexity keeps the same as before.

IV. CONCLUSION

In this paper, the structure and count problems of XOR difference probability of $(X+K)\bmod 2^n$ and $X\oplus K$ are solved completely for some special K, we proposed the best algorithm with computational complexity of $O(n)$ to solve the equivalent problem of the two operations, which is based on the "larger probability is priory to pick up" principle in savage attack, and we improved the Guess and Determine attack on SNOW1.0 of Hawkes[10] based on the algorithm, the data complexity of our attack decreases to 1/32 of Hawkes's, and the computational complexity keeps the same as before.

In fact, $(X+K)\bmod 2^n$ and $X\oplus K$ are two important operations which are used widely in cipher algorithms, so the study on the equivalent problem of the two operations has important practical values. In this paper, the best algorithm to solve the equivalent problem of the two operations is only proposed for some special K, as a future work, one can try to solve the equivalent problem of $(X+K)\bmod 2^n$ and $X\oplus K$ by extending the range of K .

REFERENCES

[1] P.Ekdahl,T.Johansson.SNOW – "a new stream cipher .Proceedings of first NESSIE workshop", Heverlee, Belgium, 2000.

[2] Ekdahl P, Johansson T. "A new version of the stream cipher Snow"[C]. In Selected Areas in Cryptography ----SAC 2002, LNCS 2595: 47-61.

[3] ETSI/SAGE."Specification of the 3GPP confidentiality and integrity algorithms UEA2&UIA2".Document 5:Design and evaluation report,version:1.0,2006. http://www.3gpp.org/ ftp/tsg_sa/WG3_Security/ TSGS3_42_ Bangalore/Docs/S3060180.zip.

[4] Biham E, Seberry J Gonzalez Neito. Py(Roo): A fast and secure stream cipher using rolling arrays[C]. ESTREAM, ECRYT Stream Cipher Project, report 2005/023, 2005.

[5] Sekar G, Paul S, Preneel B. Distinguishing attacks on the stream cipher Py[C]. ESTREAM, ECRYT Stream Cipher Project, report 2005/081, 2005.

[6] Baigneres T, Junod P, Vandenay S. "How far can we go beyond linear cryptanalysis"[C]. In Advances in Cryptology -Asiacrypt 2004, LNCS 3329: 432-450.

[7] Hiroshi Miyano. Addend Dependency of Differential/Linear Probability of Addition.IEICE Trans. Fundamentals, E81-A(1):106-109,January 1998.

[8] Alexis Warner Machado. "Differential Probability of Modular Addition with a Constant Operand". http:// eprint.iacr.org/2001/052.pdf.

[9] Zhen Bin, Guan Jie. "Differential Characteristic Probability of Added Key on Modulo 2n Operation". Journal of Electronics & Information Technology, 2009,vol31:2708-2712.

[10] P.Hawkes, G.Rose, "Guess-and-determine attacks on SNOW", proceedings of Selected Areas in Cryptography(SAC),August 2002,St John's, Newfoundland, Canada.

[11] Jin Chen-hui,Gao Hai-ying."The Cryptanalysis of two stream cipher algorithms based on chaos". Journal of Electronics,2004,34(7),1066-1070.